

Optimized Constant-Time Implementation of terSIDH

Taehun Kang, Donghoe Heo, Jeonghwan Lee, Suhri Kim, and Changmin Lee, *Member, IEEE*

Abstract—Since supersingular isogeny Diffie-Hellman (SIDH) was broken by a polynomial-time attack, several countermeasures were proposed. Among them, terSIDH has been highlighted for its high performance, yet it exposes a side-channel vulnerability. The total isogeny degree depends on the private key, causing variation in isogeny computation times. This dependency makes terSIDH susceptible to timing attacks. The ratio between the worst- and the best-case execution times of terSIDH was about 32.67 times. In this work, we propose the first constant-time terSIDH. By adding dummy operations, we reduce the dependency of isogeny computations on the private key, ensuring that the algorithm’s execution time remains constant regardless of the private key. Since such countermeasures are generally slower than their non-constant-time counterparts, we applied additional optimizations to mitigate this overhead. Moreover, we present the first C implementation of terSIDH having 128-bit security. We compared our method against CSIDH-4096, a representative isogeny-based key exchange protocol with the same security level. For key generation, the worst-case of terSIDH, CSIDH-4096, and our constant-time method take 1.77 s, 5.18 s, and 1.58 s, respectively. These results demonstrate that our proposed constant-time method is faster than the worst-case of terSIDH while also being significantly more efficient than CSIDH-4096.

Index Terms—post-quantum cryptography, isogeny, SIDH, terSIDH, constant-time

I. INTRODUCTION

ISOGENY-based cryptography is a class of post-quantum cryptography (PQC) that relies on the presumed hardness of finding isogenies between given elliptic curves. Isogeny-based cryptography was first introduced by Couveignes and independently by Rostovtsev and Stolbunov, and is today known as the CRS scheme [1]. The use of ordinary elliptic curves made the scheme inefficient and exposed it to subexponential-time attacks, which prevented it from gaining much attention. In 2011, Jao *et al.* [2] introduced the Supersingular Isogeny Diffie-Hellman (SIDH) key exchange protocol. Since SIDH is based on supersingular elliptic curves, it not only resists the attack proposed in [1], but also achieves competitive efficiency while preserving the compactness benefits of isogeny-based cryptography. This advantage led to the selection of the Supersingular Isogeny Key Encapsulation (SIKE) as a fourth-round candidate in the NIST PQC standardization process.

On the other hand, Castryck *et al.* introduced CSIDH, a key exchange protocol derived from the CRS scheme, which addresses its parameter selection problem through the use of supersingular curves [3]. The CRS framework has the advantage that CCA-secure encryption can be constructed,

enabling non-interactive key exchange. Motivated by this, the authors in [3] revisited CRS and modernized its parameter choices, improving performance by employing supersingular elliptic curves defined over $\mathbb{F}_p$. Nevertheless, because CSIDH-based algorithms have subexponential quantum complexity, they attracted less attention than SIDH, as they require larger key sizes and slower operations to achieve the same level of quantum security [4].

Meanwhile, to improve performance, SIDH was based on a slightly different problem from the general isogeny problem. First, it employs power-smooth degree isogenies so that Vélú’s formulas can be applied efficiently. Also, to derive the same shared secret key, each party computes the other party’s public key using their private key and transmits the result. This information is referred to as torsion-point information. For that reason, the CD-MM-R attack recovered the secret key of SIDH in polynomial time [5], [6], [7]. As a result, schemes such as SIKE, SETA [8], and BSIDH [9] that use torsion point information are no longer secure. Consequently, CSIDH has emerged as the most efficient isogeny-based key exchange protocol, which has renewed research interest in this direction.

Since SIDH was considered the most promising and high-performance candidate among isogeny-based cryptosystems, many studies have proposed SIDH variants with countermeasures against key-recovery attacks. As mentioned earlier, since key-recovery attacks primarily exploit torsion-point information, the first countermeasures proposed focused on masking this information. In [10], two countermeasures were introduced: M-SIDH, which conceals torsion-point information by masking it with random values, and MD-SIDH, which masks the degree of the isogeny used. However, both M-SIDH and MD-SIDH not only rely on prime fields that are reduction-unfriendly compared to the original SIDH, but also require significantly larger finite fields for the same security level. As a result, they are slower than SIDH and substantially less efficient than CSIDH [11].

On the other hand, a recently proposed countermeasure in [12] is regarded as the most efficient among SIDH variants that incorporate countermeasures against key-recovery attacks. In [12], Basso and Fouotsa proposed two countermeasures, binSIDH (binary SIDH) and terSIDH (ternary SIDH), which modify the algorithm’s structure while maintaining the SIDH framework. These protocols use artificial orientation to compute isogenies directed by the secret key. For lengths of the secret vector t, t' , binSIDH samples secret keys from $\{1, 2\}^t$, while terSIDH uses $\{0, 1, 2\}^{t'}$. As both protocols perform similar processes, terSIDH has higher security than binSIDH when using the same t . This allows terSIDH to attain the same security with smaller parameters and a highly competitive performance than binSIDH, even faster than other countermeasures.

T. Kang, D. Heo, J. Lee, and C. Lee are with the School of Cybersecurity, Korea University, Seoul 02841, South Korea (e-mail: kth9999, dong5641, hwani0814, changminlee@korea.ac.kr).

S. Kim is with the School of Mathematics, Statistics and Data Science, Sungshin Women’s University, Seoul 02844, South Korea (e-mail: suhrikim@sungshin.ac.kr).

Corresponding author: Suhri Kim (e-mail: suhrikim@sungshin.ac.kr).

However, terSIDH has a trivial problem: the total degree of evaluated isogeny depends on the secret key, resulting in variations in computational cost and ultimately making the execution time dependent on the key. Since the proposal of side-channel analysis (SCA) by Kocher *et al.* [13], resistance to such attacks has become a de facto requirement in modern cryptosystems. Moreover, because SCA resistance is mandated in standards such as FIPS 140-3, constant-time implementations against basic timing attacks are regarded as essential in the NIST PQC standardization process. Therefore, the private-key-dependent execution time of terSIDH introduces a clear risk of timing attacks, highlighting the need for an efficient constant-time implementation.

Currently, terSIDH is available only as a non-constant-time Sage prototype, which is unsuitable for constrained devices. For practical adoption, especially in IoT devices, a C implementation is essential, as such devices demand lightweight, low-level code to achieve efficiency under strict resource, memory, and energy limitations. Given the vast number of connected IoT devices requiring secure communication, this domain represents one of the most critical application areas for post-quantum cryptography. For example, the pqm4 project, which actively benchmarks and optimizes NIST PQC candidates on Cortex-M4 microcontrollers, highlights the importance of efficient and secure implementations tailored to resource-constrained IoT platforms [14]. Thus, developing an optimized constant-time C implementation of terSIDH not only enhances its security against timing attacks but also lays the groundwork for considering terSIDH in future embedded and IoT environments.

A. Our Contributions

The goal of this paper is to present an efficient constant-time implementation of terSIDH and to compare its performance with CSIDH, which is currently regarded as the most efficient isogeny-based key exchange algorithm. The main contributions of this work are as follows:

- **Identifying key-dependent components requiring constant-time implementation**

We first analyze terSIDH to identify and characterize the sources of execution-time variability that depend on the private key. Based on this analysis, we derive implementation-level countermeasures such as the insertion of dummy operations to eliminate data-dependent behavior and achieve constant-time execution.

- **Optimization of the proposed constant-time method**

We optimize the proposed constant-time technique. In a naive implementation, the constant-time version is constrained to follow the runtime of the worst-case execution of its non-constant-time counterpart. To address this, we introduce algorithmic and implementation-level improvements, thereby reducing the worst-case runtime of terSIDH by 10.7%. The detailed optimization strategies are presented in Section IV.

- **Providing the First C Implementation of terSIDH**

We present the first C implementation of terSIDH, which had previously been available only in SageMath. We

implement terSIDH in C and compare its performance against CSIDH-4096, both targeting 128-bit security. A C implementation is essential for deployment in IoT devices, where lightweight, low-level code is required to achieve efficiency within strict resource constraints. Since IoT represents a major application domain for post-quantum cryptography, demonstrating a constant-time, optimized C implementation of terSIDH not only protects against timing attacks but also shows its practicality for real-world use. Our results show that terSIDH is approximately 3.3 times faster than CSIDH, demonstrating its competitive performance among isogeny-based key exchange protocols. Details of our analysis are presented in Section V.

B. Organization

This paper is organized as follows. Section II shows the background knowledge needed to understand this paper. In Section III, we analyze the original terSIDH algorithm to discuss why terSIDH is vulnerable to timing attacks. Section IV demonstrates the proposed constant-time terSIDH. The implementation results are provided in Section V and we conclude this paper in Section VI.

II. PRELIMINARIES

In this section, we describe the concepts and structure of terSIDH to introduce the factors why terSIDH is not implemented in constant-time.

A. Montgomery arithmetic

1) *Montgomery curves*: Let K be a field of characteristic not equal to 2 or 3. An elliptic curve is a curve defined over such K and is typically given by

$$y^2 = x^3 + ax + b,$$

where $4a^3 + 27b^2 \neq 0$. Many SIDH-like protocols use Montgomery curves, which are special forms of elliptic curves because of their computational advantages. Montgomery curves are defined by the following:

$$By^2 = x^3 + Ax^2 + x,$$

where $B(A^2 - 4) \neq 0$. Since all the curves considered in this work are Montgomery curves, we will simply denote them by E from now on.

2) *Computation on Montgomery curves*: Let P and Q be points on E . Although there exists an algorithm for efficiently computing $P + Q$ using only their x -coordinates on E , performing such computations in sequence requires knowledge of the x -coordinate of $P - Q$. However, since the points P and Q change through successive isogenies, tabling the x -coordinate of $P - Q$ each time is impossible. On the other hand, since $P - P = \infty$, once the x -coordinate of a point P is known, the x -coordinate of $[2]P = P + P$ can be efficiently obtained and recursively used to compute the x -coordinate of $[n]P$ for any integer n . Therefore, the terSIDH protocol is implemented based on n -scalar multiplication, rather than point addition.

3) *Vélu's formulas*: Given a subgroup H of E and a point $P \in E$ to be evaluated, one can compute an isogeny $\phi : E \rightarrow E'$ with $\ker \phi = H$, as well as the image $\phi(P)$, by using Vélu's formulas [15]. In particular, for isogenies of degree l on Montgomery curves, Vélu's formulas can be written as follows:

$$x(\phi(P)) = x(P) \prod_{i=1}^{(l-1)/2} \left(\frac{x(P)x([i]Q) - 1}{x(P) - x([i]Q)} \right)^2,$$

where $x(\cdot)$ is x -coordinate of a point and $Q \in E$ is a generator of H . As the degree of isogeny increases, the computational cost also increases significantly.

4) *Velusqrt*: The classical computation of an l -isogeny using Vélu's formulas required $O(l)$ field operations. Bernstein *et al.* [16] introduced a quasi-square-root algorithm, often referred to as *velusqrt*. This method reduces this complexity to $\tilde{O}(\sqrt{l})$ operations by exploiting additional algebraic structure in the kernel. This improvement makes the evaluation and computation of large-degree isogenies significantly faster. The core idea is to organize the kernel points so that evaluating the relevant polynomials and isogenies can be done more efficiently, using resultants and specialized decompositions.

In *velusqrt*, the evaluation in isogeny ϕ can proceed according to the following process:

$$x(\phi(P)) = \frac{X^l \cdot h_S(1/X)^2}{h_S(X)^2},$$

where

$$X = x(P), \quad h_S(X) = \prod_{i=1}^{l-1} (X - x([s]Q))$$

in the same situation as in Vélu's formulas. This equation also demonstrates that the evaluation cost increases as the degree l increases.

B. SIDH attack

1) *SIDH*: SIDH is a key exchange protocol based on isogenies of supersingular elliptic curves. Let a and b be natural numbers, and define $A = 2^a, B = 3^b$ (with $A \approx B$). For the NIST level 1 security parameter, $a = 216$ and $b = 137$ are used. In this case, SIDH is defined over the prime $p = AB - 1$, with computations performed over the field $\mathbb{F}_{p^2}$ and $E_0 : y^2 = x^3 + 6x^2 + x$. Let (P_A, Q_A) and (P_B, Q_B) be the generators of $E_0[A]$ and $E_0[B]$ respectively.

Alice selects her secret key n_A and constructs the kernel $\ker \phi_A = \langle P_A + [n_A]Q_A \rangle$ to define the isogeny $\phi_A : E_0 \rightarrow E_A$. She then publishes the public key (E_A, R_A, S_A) , where $R_A = \phi_A(P_B)$ and $S_A = \phi_A(Q_B)$. Bob performs analogous steps to compute and publish his public key (E_B, R_B, S_B) .

Upon receiving the other party's public key, each participant uses their private key to perform a similar process. Alice computes $\phi'_A : E_B \rightarrow E_{BA}$, where $\ker \phi'_A = \langle R_B + [n_A]S_B \rangle$. Bob computes $\phi'_B : E_A \rightarrow E_{AB}$, where $\ker \phi'_B = \langle R_A + [n_B]S_A \rangle$. The shared key is the j -invariant of the resulting elliptic curve, $j(E_{BA}) = j(E_{AB})$.

2) *SIDH attack*: Computing isogenies between arbitrary elliptic curves is generally considered difficult, and the security of isogeny-based cryptosystems is based on this hard problem. However, SIDH was believed to be secure because recovering the secret isogeny from its action on torsion points was considered a hard problem, different from the general isogeny problem. Nevertheless, a series of security analyses were conducted, and the CD-MM-R attack demonstrated that the underlying problem of SIDH can be solved in polynomial time. In response, several new protocols were proposed, including *binSIDH* and *terSIDH*.

C. Artificial orientation

1) *Artificial orientation*: An artificial A -orientation of a supersingular elliptic curve E defined over $\mathbb{F}_{p^2}$ is a pair of cyclic subgroups $(G_1, G_2) \subset E[A]$ such that $G_1 \cap G_2 = \infty$. The pair $(E, (G_1, G_2))$ is then called artificial A -oriented curve. This concept allows for the definition of isogenies whose kernels are direct sums of subgroups of G_1 and G_2 , i.e., of the form $H_1 \oplus H_2$ with $H_i \subset G_i$.

In contrast to SIDH, which relies on natural subgroups determined by the structure of the torsion points, artificial orientation enables the construction of isogeny-based protocols where the subgroups used are not uniquely determined by the curve's structure. This artificiality is crucial: even if an attacker knows the image curve and its orientation, the specific isogeny path remains hidden, preventing attacks like the SIDH key recovery attack of CD-MM-R. Thus, protocols using artificial orientation, such as *terSIDH* and *binSIDH*, remain secure even in light of recent SIDH breaks.

In summary, artificial orientation provides additional randomness in the choice of subgroups, which fundamentally blocks the torsion point information leakage that the recent SIDH attacks exploited.

2) *binSIDH*: *binSIDH* (binary SIDH) is an isogeny-based key exchange protocol proposed to address the main vulnerabilities of the original SIDH protocol. By adopting artificial orientation, *binSIDH* aims to simultaneously achieve security and efficiency. The main steps of *binSIDH* are divided into parameter selection, key generation and shared key computation.

• Parameter Selection.

In *binSIDH*, a set of primes l_i, q_i for $1 \leq i \leq t$ is chosen, and $A = \prod_i l_i, B = \prod_i q_i$ are set so that the prime $p = AB - 1$ is used. Here, the values of t is determined to provide sufficient security, typically decided according to the desired security level.

Let E_0 be a supersingular elliptic curve defined over $\mathbb{F}_{p^2}$ and let P_A, Q_A and P_B, Q_B be generators of the respective torsion subgroups $E_0[A], E_0[B]$.

• Key Generation.

Alice selects a secret key $sk_A \in \{1, 2\}^t$. For $A_1 = \prod_{sk_A[i]=1} l_i, A_2 = \prod_{sk_A[i]=2} l_i$, Alice computes the isogeny $\phi_A : E_0 \rightarrow E_A$ with kernel $\langle [A_1]P_A + [A_2]Q_A \rangle$. She then sends the public key $pk_A = (E_A, (\phi_A(P_B), \phi_A(Q_B)))$.

Similarly, Bob selects his secret key $sk_B \in \{1, 2\}^t$, performs the same process, and sends the public key $pk_B = (E_B, (\phi_B(P_A), \phi_B(Q_A)))$.

- **Shared Key Computation.**

Upon receiving Bob's public key $(E_B, (\phi_B(P_A), \phi_B(Q_A)))$, Alice first checks whether $(\phi_B(P_A), \phi_B(Q_A))$ forms an artificial orientation on E_B . If so, she uses her secret key sk_A to compute an isogeny $\phi'_A : E_B \rightarrow E_{BA}$ and derives the shared key $j(E_{BA})$.

Likewise, Bob, upon receiving Alice's public key $(E_A, (\phi_A(P_B), \phi_A(Q_B)))$ and using his secret key sk_B , follows the same procedure and compute the shared key $j(E_{AB})$.

3) *terSIDH*: *terSIDH* (ternary SIDH) employs an algorithmic structure similar to that of *binSIDH*, but differs in that the secret key is selected from $\{0, 1, 2\}^t$. Therefore, when using the same security parameters, *terSIDH* achieves approximately $\log_2 3$ times higher security than *binSIDH*, allowing the parameter size to be significantly reduced for the same level of security.

During the key generation process of *binSIDH*, the kernel of Alice's isogeny ϕ_A is defined as

$$\ker \phi_A = \langle [A_1]P + [A_2]Q \rangle,$$

that is, A_1 and A_2 are products of all distinct primes in A, B .

In contrast, *terSIDH* uses the secret key from $\{0, 1, 2\}^t$, leading to a slightly different structure. In *terSIDH*, the kernel of Alice's key generation isogeny is given by

$$\ker \phi_A = \langle [A_1]P + [A_2]Q \rangle,$$

where

$$A_0 = \prod_{sk_A[i]=0} l_i, A_1 = A_0 \prod_{sk_A[i]=1} l_i, A_2 = A_0 \prod_{sk_A[i]=2} l_i.$$

In *terSIDH*, the number of isogeny computations varies depending on the values of A_0 . Specifically, as A_0 become larger, the algorithm uses fewer isogeny computations; conversely, as A_0 becomes smaller, more isogeny computations are required. Although this behavior improves efficiency, it also creates a vulnerability: since the number of computations varies with the secret key, the execution time may reveal information about the key, making the protocol susceptible to timing attacks.

III. TIMING SIDE-CHANNEL VULNERABILITIES IN THE ORIGINAL *TERSIDH*

In this section, we examine the original *terSIDH* algorithm and investigate the sources of its execution time variation. Since Alice and Bob follow similar procedures, we focus exclusively on Alice's side of the algorithm.

A. Analysis of the Original Algorithm

The parameter generation process depends only on the parameter size. Therefore, the functions in which execution time variation occurs are the key generation function and the shared secret generation function. Since the shared secret computation process is similar to the key generation function,

Algorithm 1 Key generation of *terSIDH* [12]

Parameter: $A = \prod_{i=1}^t l_i$

Input: $sk_A \in \{0, 1, 2\}^t$, $P, Q, R, S \in E$

Output: $pk_A = (E_A, R, S)$

```

1:  $A_1 \leftarrow 1, A_2 \leftarrow 1$ 
2: for  $i$  from 1 to  $t$  do
3:   if  $sk_A[i] = 1$  then
4:      $A_1 \leftarrow l_i A_1$ 
5:   else if  $sk_A[i] = 2$  then
6:      $A_2 \leftarrow l_i A_2$ 
7:   else
8:      $A_1 \leftarrow l_i A_1, A_2 \leftarrow l_i A_2$ 
9:   end if
10: end for
11:  $\ker \phi'_A \leftarrow \langle [A_1]P \rangle$ 
12: Compute  $\phi'_A : E \rightarrow E'$  with  $\ker \phi'_A$ 
13:  $Q \leftarrow \phi'_A(Q), R \leftarrow \phi'_A(R), S \leftarrow \phi'_A(S)$ 
14:  $\ker \phi''_A \leftarrow \langle [A_2]Q \rangle$ 
15: Compute  $\phi''_A : E' \rightarrow E_A$  with  $\ker \phi''_A$ 
16:  $R \leftarrow \phi''_A(R), S \leftarrow \phi''_A(S)$ 
17: return  $pk_A = (E_A, R, S)$ 
```

we concentrate on analyzing the structure of the key generation function in detail. Algorithm 1 presents the key generation algorithm of the original *terSIDH*.

Given a kernel, isogenies can be constructed using Vélú's formulas. For small primes l_i with $A = \prod_{i=1}^t l_i$, when the secret key $sk_A \in \{0, 1, 2\}^t$ is chosen, we have $A_0 = \prod_{sk_A[i]=0} l_i$. Suppose that the generators of $E[A]$ are P and Q , then $\ker \phi_A = \langle [A_1]P + [A_2]Q \rangle$, where $A_1 = A_0 \prod_{sk_A[i]=1} l_i, A_2 = A_0 \prod_{sk_A[i]=2} l_i$. Here, the degree of isogeny ϕ_A is given by the number of elements in $\langle [A_1]P + [A_2]Q \rangle$, that is, $A_1 A_2 / A_0^2$. In other words, the degree of ϕ_A decreases as the value of A_0 increases.

In *terSIDH*, Montgomery curves are used to achieve fast scalar multiplication. On Montgomery curves, in order to compute $[A_1]P + [A_2]Q$, the x -coordinate of $[A_1]P - [A_2]Q$ must be known. However, since A_1 and A_2 depend on the private key, their values cannot be precomputed and used directly in the algorithm. Therefore, in the reference implementation of *terSIDH*, the kernel is decomposed into two isogenies with kernel $\ker \phi'_A = \langle [A_1]P \rangle$ and $\ker \phi''_A = \langle [A_2]Q \rangle$, and the isogeny is computed as a composition $\phi_A = \phi''_A \circ \phi'_A$. By ensuring that the degrees of ϕ'_A and ϕ''_A are coprime, the two isogenies do not interfere with each other, and the overall result remains equivalent to the original operation. Splitting the computation into two separate isogenies also leads to a faster implementation than computing ϕ_A in a single step, and is therefore adopted in practice.

To obtain an isogeny of degree $A = \prod_{i=1}^t l_i$, one must perform t isogeny computations, each with degree l_i . Suppose the order of a point P is A . If the order of P is divisible by l_j , to compute an isogeny of degree l_j one would need to compute $[A/l_j]P$, which can be extremely time-consuming if performed directly for every l_j . To address this, existing implementations multiply the unused primes in advance and

Algorithm 2 Isogeny computation in [12]**Input:** integer A , elliptic curve $E, P \in E$ **Output:** isogeny ϕ

```

1:  $n \leftarrow A, \phi \leftarrow [1], \phi' \leftarrow [1]$ 
2: for  $l^e$  in  $A.\text{factor}$  do
3:    $P \leftarrow \phi'(P), \phi' \leftarrow [1]$ 
4:    $n \leftarrow n/l^e, \ker \phi' \leftarrow \langle [n]P \rangle, \phi \leftarrow \phi' \circ \phi$ 
5: end for
6: return  $\phi : E \rightarrow E'$ 

```

perform the above process only for the remaining primes.

Let us consider the situation where we compute $\langle [A_1]P \rangle$. In this case, the order of $[A_1]P$ is $A/A_1 = A_2/A_0$. Therefore, after computing $P' = [A_1]P$, one then computes $\langle [A_2/l_j]P' \rangle$ to reduce the number of operations. Furthermore, once point P' passes through the isogeny $\phi_{A_{l_i}}$ of degree l_j , the degree l_j no longer affects the order of $\phi_{A_{l_i}}(P')$. By taking $A_2 \leftarrow A/l_j$ and sequentially computing $\langle [A_2]P \rangle$, the process can be optimized.

As a result, the original terSIDH implementation uses this method to significantly reduce the computational workload compared to a naive single-step approach.

B. Analysis of Execution Time Differences

1) *Variation Due to the Secret Key:* According to the previous definition, the order of $\ker \phi_A$ is A/A_0 , where A_0 is the product of the primes corresponding to zeros in the secret key sk_A . As shown in Algorithm 1, the more zeros present in the secret key sk_A , the larger the value of A_0 becomes, thereby reducing the order of $\ker \phi_A$ and subsequently decreasing the degree of ϕ_A . ϕ_A can be written as $\phi_A = \phi_{A_{l_1}} \circ \dots \circ \phi_{A_{l_t}}$. As the degree decreases, the number of small isogenies to be composited decreases, reducing the overall number of isogeny computations. Therefore, the number of zeros in the secret key leads to variation in execution time.

Furthermore, instead of computing $\ker \phi_A = \langle [A_1]P + [A_2]Q \rangle$, when computing isogenies in the form $\ker \phi'_A = \langle [A_1]P \rangle$ and $\ker \phi''_A = \langle [A_2]Q \rangle$, after calculating the isogeny ϕ_A , the point Q must also be evaluated under ϕ_A , that is, $\langle [A_2]\phi'_A(Q) \rangle$ must be used as the second kernel. In other words, additional evaluation of Q under ϕ_A is required. According to Vélu's formulas for Montgomery curves, each evaluation operation increases the computational cost depending on the degree of the isogeny. Therefore, the degree of $\phi'_A(Q)$ is $A/A_1 = A_2/A_0$ and the number of such evaluations also affects the total computational cost depending on the secret key.

2) *Variation Due to Scalar Multiplication:* The algorithm 2 shows the isogeny computation algorithm in the original terSIDH.

According to Algorithm 2, the calculation of $n \leftarrow n/l_i$ is performed before scalar multiplication. If the primes are ordered in descending order, the rate of decrease for the scalar is initially low and then gradually increases. If the primes are ordered in ascending order, the decrease rate is high at first but gradually decreases thereafter. Therefore, the total number of

isogeny computations is likely to be higher when the primes are arranged in descending order compared to ascending order.

In previous terSIDH implementations, the order in which the prime degrees are processed can affect the computation time for isogenies of each prime degree. At this stage, the time required for scalar multiplication depends on the order of the primes: when the scalar to be multiplied is large, the computation takes longer; as the scalar becomes smaller, the computation time decreases accordingly.

In particular, if the primes are arranged in ascending order, scalar multiplication with small primes is computed at the beginning, leading to a gradual decrease in computation time. However, as larger primes are processed later, there is a sudden reduction in computation time for scalar multiplication. Conversely, when the primes are arranged in descending order, scalar multiplication with large scalars is performed first, resulting in a rapid decrease in computation time at the early stages, followed by a relatively gentle reduction in later stages.

On the other hand, the time for evaluation operations and Vélu's formulas tends to increase as the size of the primes grows. The total number of evaluations and Vélu operations is unaffected by the ordering of the primes, but the overall running time for scalar multiplications is maximized when the primes are ordered in descending order. Therefore, comparing the total execution times, it can be seen that ascending order generally results in longer total computation times than descending order.

When analyzing the actual execution patterns, arranging the primes in ascending order produces an execution time curve that increases gradually before dropping sharply at the end. Figure 1 illustrates the execution-time curve when the primes are arranged in ascending order. The x -axis represents n -th isogeny computation along the progression of computations, while the y -axis denotes the execution time. For example, at $x = 30$ the algorithm is performing the 30th isogeny, and the y -axis indicates the corresponding execution time.

In contrast, arranging the primes in descending order yields a steep drop at the beginning, followed by a slower decline. In both cases, the sequence and pattern of operations are highly regular and repetitive, and the total number of operations at each stage is determined by the secret key. As a result, the timing pattern tends to exhibit distinct trends depending on the secret key. This property creates a potential vulnerability that may enable attackers to infer secret information through timing analysis. Figure 2 show the timing patterns of isogeny computation when primes are in descending order. The meanings of the x - and y -axes are the same as in Figure 1.

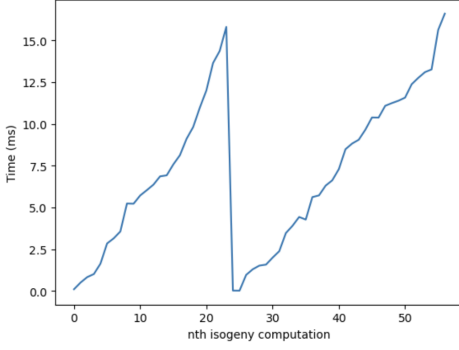


Fig. 1. Timing pattern of isogeny computation when primes are arranged in ascending order

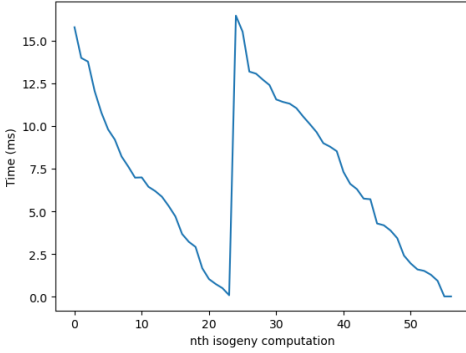


Fig. 2. Timing pattern of isogeny computation when primes are arranged in descending order

IV. PROPOSED CONSTANT-TIME TERSIDH

In this section, based on the analysis presented in Section 3, we propose an improved algorithm of isogeny computation that achieves constant-time execution by introducing dummy operations.

A. Removing Key-Dependent Variations

To reduce the execution time variation caused by the number of zeros in the secret key, we add dummy operations to the key generation algorithm. Specifically, we define a dummy value $A_0 = \prod_{sk[i]=0} l_i$ and compute dummy isogenies accordingly, as illustrated in Algorithm 3.

In Algorithm 3, for all i such that $sk_A[i] = 0$, the degree of ϕ_A is divided by l_i and the degree of dummy isogeny ϕ_C is multiplied by the same factor. Therefore, regardless of the number of zeros in the secret key, isogenies of degree l_i are calculated for all l_i . By distinguishing between ϕ_A and ϕ_C and ensuring that the total number of isogeny computations remains the same, we achieve a constant-time implementation from this perspective. The dummy isogeny ϕ_C is stored together with ϕ_A and used during the evaluation process to guarantee a constant execution time.

For Alice's public key E_A , the isogeny $\phi_A : E \rightarrow E_A$ depends on the secret key and thus varies in degree. Consequently, we perform dummy operations when computing $\phi_A(R)$, $\phi_A(S)$, adding the results of dummy evaluation

Algorithm 3 Proposed constant-time key generation

Parameter: $A = \prod_{i=1}^t l_i$

Input: $sk_A \in \{0, 1, 2\}^t$, $P, Q, R, S \in E$

Output: $pk_A = (E_A, R, S)$

```

1:  $A_0 \leftarrow 1$ ,  $A_1 \leftarrow 1$ ,  $A_2 \leftarrow 1$ ,  $C \leftarrow Q$ ,  $R_C \leftarrow R$ ,  $S_C \leftarrow S$ 
2: for  $i$  from 1 to  $t$  do
3:   if  $sk_A[i] = 1$  then
4:      $A_1 \leftarrow l_i A_1$ ,  $A_0 \leftarrow l_i A_0$ 
5:   else if  $sk_A[i] = 2$  then
6:      $A_2 \leftarrow l_i A_2$ ,  $A_0 \leftarrow l_i A_0$ 
7:   else
8:      $A_1 \leftarrow l_i A_1$ ,  $A_2 \leftarrow l_i A_2$ 
9:   end if
10: end for
11:  $\ker \phi'_A \leftarrow \langle [A_1]P \rangle$ 
12: Compute  $\phi'_A : E \rightarrow E'$  with  $\ker \phi'_A$ 
13:  $Q \leftarrow \phi'_A(Q)$ ,  $R \leftarrow \phi'_A(R)$ ,  $S \leftarrow \phi'_A(S)$ 
14:  $\ker \phi''_A \leftarrow \langle [A_2]Q \rangle$ 
15: Compute  $\phi'_A : E' \rightarrow E_A$  with  $\ker \phi''_A$ 
16:  $R \leftarrow \phi''_A(R)$ ,  $S \leftarrow \phi''_A(S)$ 
17:  $\ker \phi_C \leftarrow \langle [A_0]C \rangle$ 
18: Compute  $\phi_C : E \rightarrow E_C$  with  $\ker \phi_C$ 
19: (Dummy) Compute  $\phi_C(R_C)$ ,  $\phi_C(S_C)$ 
20: return  $pk_A = (E_A, R, S)$ 

```

$\phi_C(R)$, $\phi_C(S)$ as necessary to ensure a uniform computation time.

As mentioned in Section 3.B, the isogeny ϕ''_A uses $\langle [A_2]\phi'_A(Q_A) \rangle$ as its kernel, so we must also evaluate $\phi'_A(Q_A)$. The computation time for this step depends on the order of $[A_1]P$. Therefore, in practice, it is necessary to perform additional evaluations, such as $\phi''_A(P)$, to achieve a constant-time implementation. However, as will be described in Section 4.C, our proposed method eliminates the need for these additional computations.

B. Removing Scalar Multiplication-Dependent Variations

In Algorithm 2, the prime factors are arranged in ascending order, so the smaller primes are evaluated first when computing isogenies. Due to this ordering, smaller primes are always processed first, which leads to a bias in the pattern of scalar multiplication operations. Therefore, to hide such timing patterns in isogeny computations, we randomized the order in which the primes are evaluated. Figure 3 shows the timing pattern of the random case, and the detailed results are covered in a later section.

C. Additional Optimized Implementation

In terSIDH, isogenies are computed using kernels of the form $\langle [A_1]P + [A_2]Q \rangle$. However, it is well known that the x -coordinate of $P+Q$ cannot be efficiently computed without the x -coordinate of $P-Q$ on Montgomery curves. For this reason, in the original terSIDH, isogenies are computed with $\phi_A = \phi''_A \circ \phi'_A$, for $\ker \phi'_A = \langle [A_1]P \rangle$ and $\ker \phi''_A = \langle [A_2]Q \rangle$, which makes the original implementation faster. However, since the

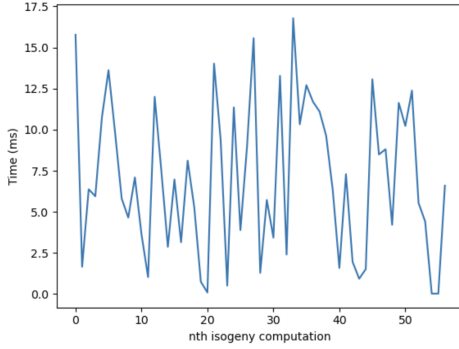


Fig. 3. Timing pattern of isogeny computation when primes are arranged randomly

addition of dummy operations increases the computation cost for each point, computing $\langle [A_1]P + [A_2]Q \rangle$ in a single step can potentially be more efficient.

The implementation proceeds as follows. Since $[A_1]P + [A_2]Q$ cannot be computed using only x -coordinates, we first compute the corresponding y -coordinate by substituting it into the equation of the elliptic curve. Using this, the conventional point addition algorithm is applied to obtain $T = [A_1]P + [A_2]Q$. Compared to the previous method, where operations were performed for each of P, Q , and dummy point C , this approach requires computation only for the two points T, C . Our experimental comparison confirmed that using $[A_1]P + [A_2]Q$ directly as a kernel yields better performance.

V. IMPLEMENTATIONS

In this section, we present the first C implementation of terSIDH and evaluate its performance against CSIDH, which is considered the most efficient isogeny-based key exchange protocol to date.

The original implementation of terSIDH was available only in SageMath. Although SageMath is useful for understanding the algorithmic structure, it is slow and makes it difficult to compare terSIDH with other cryptographic protocols. Therefore, we implemented terSIDH in C. Furthermore, we incorporated both constant-time techniques and additional optimizations as described above. Additionally, we ported terSIDH to C with reference to the implementation of Kim [11] and further incorporated the constant-time and optimization techniques originally developed in SageMath.

We then compared our implementation with CSIDH-4096, which is demonstrated to have a 128-bit quantum security level. For CSIDH-4096, we used [4]. For a fair comparison, we also used a constant-time implementation of CSIDH. Specifically, we adopted the method of Onuki *et al.*, which is known to be the fastest constant-time version of CSIDH. The details of this constant-time CSIDH method can be found in [17].

A. Implementation results

Table I shows the performance difference of key generation algorithms depending on their prime orders. It is evident that this approach effectively confuses the timing patterns.

Additionally, randomizing the order reduces the execution time compared to the conventional method, where the primes are processed strictly in descending order. We tested all our implementations on a single core of an Intel Core i7-8700K (Coffee Lake) processor, clocked at 3,700 MHz and 16GB of RAM.

TABLE I
EXECUTION TIME COMPARISON BY THE ORDER OF PRIMES IN SAGEMATH (MS)

protocol	normal	0	1	2	0,1	0,2	1,2
original	2260.9	148.1	4453.0	4838.2	1813.9	1989.1	3653.9
random	2226.5	149.1	4397.3	4691.4	1744.8	1988.0	3585.0
reverse	2175.3	148.6	4130.9	4520.8	1700.6	1945.6	3487.7

Table II compares the key generation time of conventional binSIDH and terSIDH implementations, as well as our constant-time terSIDH algorithms, according to different types of private keys. To highlight the differences, we provide a comparison between the algorithm that incorporates the optimization technique and its original version without the technique. For each case, we ensured that private keys always start with 0, 1, 2 so that subsequent parts of the private keys vary across tests; this is to ensure that the isogeny is not trivial.

TABLE II
EXECUTION TIME COMPARISON IN SAGEMATH (MS)

protocol	normal	0	1	2	0,1	0,2	1,2
binSIDH	12515.9	-	15382.6	16759.9	-	-	-
terSIDH	2260.9	148.1	4453.0	4838.2	1813.9	1989.1	3653.9
constant	5221.2	5223.0	5227.9	5220.1	5223.8	5221.2	5221.2
Ours	4383.8	4387.3	4391.0	4390.0	4392.4	4389.8	4395.9

In theory, binSIDH should not exhibit any timing differences depending on the type of private key. However, in previous implementations, as discussed in Section 3.B.1, the number of '2's included in the private key caused the variation in implementation, not algorithmically. Although it can be easily implemented in constant-time, the required time is too long compared to that of terSIDH, so only the performance of the previous implementation was measured without further analysis.

For terSIDH, as analyzed, significant variations in execution time were observed depending on the degree of bias in the private key. When the value of A_0 increases, the degree of isogeny calculated decreases. Conversely, as the number of '2's in the private key increases, the computation cost increases. Therefore, the execution time was the longest when all private key entries were set to '2', and the shortest when all entries were '0'. We measured the maximum-to-minimum execution time ratio for terSIDH, which was as high as 32.67, clearly showing the extent of this timing variability.

In contrast, our proposed algorithm achieved a ratio of just about 1.00 across all private key types, demonstrating that constant-time execution was successfully achieved. Furthermore, we achieved better performance than the worst-case scenario of the original terSIDH implementation, 1774.4

ms. This shows that our method improved efficiency without incurring a performance penalty while preserving the security benefits of constant-time implementation.

TABLE III
EXECUTION TIME COMPARISON IN C (MS)

protocol	normal	0	1	2	0,1	0,2	1,2
terSIDH	793.9	32.6	1605.2	1774.4	633.5	718.8	1308.8
Ours	1580.3	1580.0	1580.0	1580.0	1580.0	1579.8	1579.9
CSIDH-4096	5177.7	-	-	-	-	-	-

VI. CONCLUSION

Summary. In this paper, we present a constant-time implementation of terSIDH, which is one of the countermeasures against the CD-MM-R attack. A drawback of terSIDH is that its execution time varies since the degree of isogeny depends on the private key. To address this issue, we achieved a constant-time implementation by introducing dummy operations. Additionally, we further improved the algorithm, achieving a shorter execution time compared to the worst case of the previous implementation. We also improved the implementation, such as the order of primes sorting and optimization methods.

Results. We indicate that the constant-time implementation of terSIDH eliminates observable key-dependent timing variations. In our C implementation, whereas the original terSIDH implementation exhibited a maximum-to-minimum execution-time ratio of approximately 32.67 across different types of private key classes, our constant-time variant reduces this ratio to nearly 1.0, yielding a fully uniform timing profile. Moreover, the optimized version maintains constant-time characteristics, while improving the worst-case performance about 11.0% compared to the original terSIDH. Our implementation demonstrates that terSIDH possesses competitive performance with existing isogeny-based protocols. For comparison, CSIDH-4096, which targeted NIST level 1 security, requires substantially larger parameters and exhibits markedly slower key generation than our terSIDH at the same security level.

Future work. At present, the state-of-the-art isogeny-based key exchange protocol is terSIDH-hyb, in which Alice and Bob execute both terSIDH-like and SIDH-like isogeny evaluations. The constant-time countermeasures and implementation optimizations developed in this paper are immediately applicable to the terSIDH-like component of terSIDH-hyb. This is expected to strengthen resistance to timing side-channel attacks as well as its performance.

REFERENCES

- [1] A. Childs, D. Jao, and V. Soukharev, "Constructing elliptic curve isogenies in quantum subexponential time," *Journal of Mathematical Cryptology*, vol. 8, no. 1, pp. 1–29, 2014.
- [2] D. Jao and L. De Feo, "Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies," in *International workshop on post-quantum cryptography*. Springer, 2011, pp. 19–34.
- [3] W. Castryck, T. Lange, C. Martindale, L. Panny, and J. Renes, "Csidh: an efficient post-quantum commutative group action," in *International conference on the theory and application of cryptology and information security*. Springer, 2018, pp. 395–427.

- [4] J. Chávez-Saab, J.-J. Chi-Domínguez, S. Jaques, and F. Rodríguez-Henríquez, "The scale of csidh: sublinear vélu quantum-resistant isogeny action with low exponents," *Journal of Cryptographic Engineering*, vol. 12, no. 3, pp. 349–368, 2022.
- [5] W. Castryck and T. Decru, "An efficient key recovery attack on sidh," in *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 2023, pp. 423–447.
- [6] L. Maino, C. Martindale, L. Panny, G. Pope, and B. Wesolowski, "A direct key recovery attack on sidh," in *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 2023, pp. 448–471.
- [7] D. Robert, "Breaking sidh in polynomial time," in *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 2023, pp. 472–503.
- [8] L. De Feo, C. Delpech de Saint Guilhem, T. B. Fouotsa, P. Kutas, A. Leroux, C. Petit, J. Silva, and B. Wesolowski, "Séta: Supersingular encryption from torsion attacks," in *International conference on the theory and application of cryptology and information security*. Springer, 2021, pp. 249–278.
- [9] C. Costello, "B-sidh: supersingular isogeny diffie-hellman using twisted torsion," in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2020, pp. 440–463.
- [10] T. B. Fouotsa, T. Moriya, and C. Petit, "M-sidh and md-sidh: countering sidh attacks by masking information," in *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 2023, pp. 282–309.
- [11] S. Kim, "Performance and efficiency evaluation of m-sidh," *IEEE Access*, 2024.
- [12] A. Basso and T. B. Fouotsa, "New sidh countermeasures for a more efficient key exchange," in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2023, pp. 208–233.
- [13] P. C. Kocher, "Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems," in *Annual international cryptology conference*. Springer, 1996, pp. 104–113.
- [14] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, "pqm4: Testing and benchmarking nist pqc on arm cortex-m4," 2019.
- [15] J. Vélu, "Isogénies entre courbes elliptiques," *Comptes-Rendus de l'Académie des Sciences*, vol. 273, pp. 238–241, 1971.
- [16] D. J. Bernstein, L. De Feo, A. Leroux, and B. Smith, "Faster computation of isogenies of large prime degree," *Open Book Series*, vol. 4, no. 1, pp. 39–55, 2020.
- [17] H. Onuki, Y. Aikawa, T. Yamazaki, and T. Takagi, "A constant-time algorithm of csidh keeping two points," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. 103, no. 10, pp. 1174–1182, 2020.

Taehun Kang received the B.S. degree in Mathematics and the M.S. degree in Information Security from Korea University, in 2023 and 2025, respectively, and he is currently pursuing the Ph.D. degree with the Graduate School of Information Security, Korea University. His research interests include post-quantum cryptography.

Donghoe Heo received the B.S. degree in mathematics from Hanyang University, in 2019, and the Ph.D. degree from the Graduate School of Information Security, Korea University, in 2025. His research interests include post-quantum cryptography, especially isogeny-based cryptosystems, and quantum algorithms.

Jeonghwan Lee received the B.S. degree in artificial intelligence cyber security and the M.S. degree in Information Security from Korea University, in 2023 and 2025, respectively. He is currently pursuing the Ph.D. degree with the Graduate School of Information Security, with a focus on post-quantum cryptography, especially isogeny-based cryptosystems, and side-channel analysis.

Suhri Kim received the B.S. degree in mathematics and the M.S. degree in Information Security from Korea University, in 2014 and 2016, respectively, and the Ph.D. degree from the Graduate School of Information Security, Korea University, in 2020. She is currently an Associate Professor with the School of Mathematics, Statistics and Data Science, Sungshin Women's University. Her research interests include post-quantum cryptography and efficient computations for isogeny-based cryptosystems.

Changmin Lee received the B.S. degree in mathematics from Seoul National University, South Korea, in 2012, and the Ph.D. degree in mathematics from Seoul National University in 2017. From 2017 to 2018, he was a research fellow with the Research Institute of Basic Science, Seoul National University. From 2018 to 2020, he was a postdoctoral researcher at ENS de Lyon, France. From 2020 to 2025, he was an assistant professor at the Korea Institute for Advanced Study. He is currently an Associate Professor with the Graduate School of Cybersecurity, Korea University, Seoul. His research interests include cryptography, post-quantum cryptography, and mathematical aspects of security.